

微服务架构规范

技术规范文件

保险公司 技术开发部
内部文档 · 仅供授权人员查阅

目录

- 1. 概述
- 2. 服务拆分原则
 - 2.1 按业务领域拆分（DDD 限界上下文）
 - 2.2 拆分原则优先级
- 3. 技术选型
- 4. 服务通信规范
 - 4.1 通信模式选择
 - 4.2 API 规范
 - 4.3 服务间调用超时与重试
- 5. 服务治理
 - 5.1 服务注册与发现
 - 5.2 限流与降级
 - 5.3 配置管理
- 6. 事务一致性
 - 6.1 跨服务事务策略
 - 6.2 Saga 编排示例（承保流程）
- 7. Kubernetes 部署规范
- 8. 版本管理

微服务架构规范

1. 概述

本文档定义保险技术平台的微服务架构标准，包括服务拆分原则、框架选型、服务治理、通信模式等内容。参考 Spring Cloud / Kubernetes 生态最佳实践，融合蚂蚁集团 SOFAShark 等大厂经验。

2. 服务拆分原则

2.1 按业务领域拆分（DDD 限界上下文）

领域	限界上下文	服务候选
产品域	产品定义、费率、条款	product-service
承保域	投保、核保、出单	underwriting-service, risk-engine
保单域	保单生命周期	policy-service
保全域	变更、复效、退保	servicing-service
理赔域	报案、查勘、定损、理算	claim-service, assessment-service
续期域	缴费、宽限期、失效	renewal-service
再保域	分入分出、合同、账单	reinsurance-service
渠道域	渠道管理、佣金、协议	channel-service, commission-service
收付域	收付费、对账	payment-service, reconciliation-service
客户域	客户信息、认证	customer-service
用户域	员工、代理人账号	user-service

2.2 拆分原则优先级

1. 业务职责 — 同一限界上下文内的功能聚合
2. 变更频率 — 高频变更与低频变更分离
3. 数据独立性 — 每个服务拥有独立的数据库（Database per Service）
4. 团队结构 — 逆康威定律，匹配团队组织
5. 性能要求 — 高吞吐服务独立部署和扩缩

3. 技术选型

组件	技术栈	说明
开发框架	Spring Boot 3.x / Spring Cloud 2023	Java 17+
注册中心	Nacos / Consul	支持 AP + CP 模式
配置中心	Nacos Config / Apollo	配置热更新
网关	Spring Cloud Gateway / Kong	路由、鉴权、限流
RPC 框架	OpenFeign + Resilience4j	声明式调用 + 熔断降级
消息队列	RocketMQ / Kafka	异步解耦、最终一致性
容器编排	Kubernetes + Istio	服务网格
可观测	OpenTelemetry + Prometheus + Grafana	三支柱

4. 服务通信规范

4.1 通信模式选择

场景	推荐模式	协议
同步查询	RESTful API / gRPC	HTTP/2, Protobuf
命令写入	异步消息 + 事件驱动	RocketMQ / Kafka
跨域事务	Saga（编排模式）	消息 + 本地事务表
状态变更通知	CloudEvents / Domain Events	消息队列
文件传输	对象存储 + 消息通知	S3/MinIO + MQ

4.2 API 规范

- URL 范式: `/ {version} / {domain} / {resource}`
- 版本策略: URL Path Versioning (`/v1/policies`)
- 响应格式: 统一 JSON 结构

```
{
  "code": 0,
  "message": "success",
  "data": {},
  "traceId": "xxx"
}
```

- 错误码: 领域编码(2位) + 业务码(4位)，如 010001

4.3 服务间调用超时与重试

调用类型	超时	重试次数	熔断阈值
同步查询	3s	1	50% 失败/10s
同步写入	5s	0	30% 失败/10s
异步消息	7天重试	16次(指数退避)	-

5. 服务治理

5.1 服务注册与发现

- 所有服务通过 Nacos 注册，健康检查间隔 5s
- 关键服务设置 weighted load balancing
- 金丝雀发布通过 metadata 标签路由

5.2 限流与降级

- 网关层: 全局限流（令牌桶）
- 服务层: 接口级限流（Sentinel / Resilience4j）
- 降级策略: 返回兜底缓存数据或友好错误提示
- 熔断恢复: 半开状态，5s 后尝试放行

5.3 配置管理

- 环境隔离: dev / test / staging / prod
- 配置变更: Nacos 监听 + Apollo 灰度发布
- 敏感配置: 集成密钥管理服务，DB 加密存储

6. 事务一致性

6.1 跨服务事务策略

- 强一致（极少场景）：尽量规避，仅限同一服务内本地事务
- 最终一致（默认）：Saga + 本地消息表 + 定时对账
- 补偿机制：每个跨域写操作设计对应的补偿/回滚接口

6.2 Saga 编排示例（承保流程）

□□ → □□(□□) → □□(□□) → □□□□(□□)
□□：□□□□→□□；□□□□→□□□□Saga

7. Kubernetes 部署规范

配置项	规范值
请求资源	CPU: 500m, Memory: 1Gi
限制资源	CPU: 2, Memory: 4Gi
就绪探针	HTTP GET /actuator/health/readiness, 间隔 10s
存活探针	HTTP GET /actuator/health/liveness, 间隔 30s
HPA 策略	CPU > 70% 或 Memory > 80% 触发扩容
Pod 干扰预算	minAvailable: 1 (>= 3副本) 或 50%

8. 版本管理

- 语义版本：MAJOR.MINOR.PATCH
- 镜像标签：{service-name}:{semver}-build.{build-number}
- API 版本：向后不兼容变更升 MAJOR，新增字段升 MINOR